

Common Software Architecture Design Patterns

Layered (N-Tier) Architecture

- Components are organized into horizontal layers,
- Each with a specific responsibility.
- Each layer interacts only with the layer directly below or above it,
- Promotes separation of concerns, modularity, and maintainability.

Key Characteristics

Separation of Concerns – Each layer has a distinct responsibility.

Abstraction – Layers hide implementation details from other layers.

Dependency Flow – Typically, higher layers depend on lower layers, but not vice versa.

Testability – Layers can be tested independently.

Scalability – Layers can be scaled or modified without affecting others.

Common Layers in Layered Architecture

Layer	Responsibility	Example Components
Presentation Layer (UI)	Handles user interaction	Web UI, Mobile App, API Controllers
Business Logic Layer (Service Layer)	Implements business rules	Services, Domain Logic, Use Cases
Data Access Layer (Persistence Layer)	Manages data storage & retrieval	Repositories, DAOs, ORM (Hibernate, Entity Framework)
Database Layer	Stores and retrieves data	MySQL, PostgreSQL, MongoDB

Common Layers in Layered Architecture

(Note: Some variations may include additional layers like API Layer, Domain Layer, or Cross-Cutting Layers for logging, security, etc.)

Advantages & Disadvantages

Advantages

✓ **Modularity** – Easier to maintain and update individual layers.

✓ **Reusability** – Business logic can be reused across different UIs (Web, Mobile, API).

✓ **Testability** – Layers can be mocked and tested in isolation.

✓ **Separation of Concerns** – Clear boundaries between UI, business logic, and data access.

✓ **Team Collaboration** – Different teams can work on different layers simultaneously.

Disadvantages

✗ **Performance Overhead** – Multiple layers can introduce latency (e.g., passing data through each layer).

✗ **Tight Coupling Risk** – Poor design can lead to dependencies between layers.

✗ **Complexity** – Over-engineering for simple applications.

-
- **When to Use Layered Architecture?**
 - Enterprise Applications (ERP, CRM systems)
 - Web Applications with clear separation between UI, logic, and data
 - Long-term Projects requiring maintainability and scalability
 - **Alternatives to Layered Architecture**
 - Microservices (Decoupled services instead of layers)
 - Hexagonal (Ports & Adapters) (Focuses on domain-centric design)
 - Event-Driven Architecture (Components communicate via events)

Example

```
▪ // Presentation Layer (Controller)

▪ @RestController

▪ public class UserController {

▪     @Autowired

▪     private UserService userService; // Business Layer dependency

▪

▪     @GetMapping("/users/{id}")

▪     public User getUser(@PathVariable Long id) {

▪         return userService.getUserById(id);

▪     }

▪ }

▪

▪ // Business Layer (Service)

▪ @Service
```